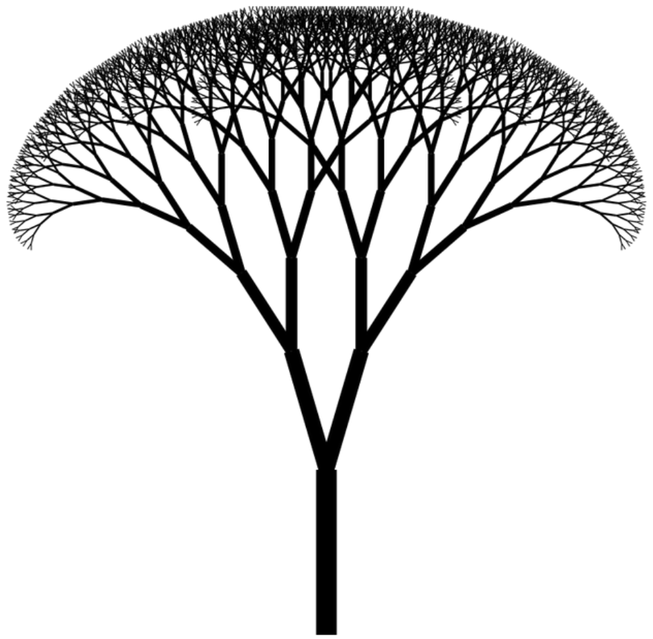
Presentation

Fractal trees

In what ways does the shape resemble a tree?

- The tree has a main trunk that splits into branches
- the branches continue to divide into smaller branches and it shows a repeating pattern



In what ways is it a simplified / unrealistic tree?

- Each of the branches are evenly spaced out and have the exact same angles
- Trees do not grow like this, they grow irregularly while this one is perfectly even and symmetrical

Fractal structure

A fractal tree is a structured with a pattern of branches that repeat themselves at a smaller scale. Each branch is similar to the previous one. Each tree branch splits into smaller branches and those branches split into smaller branches. Fractal trees allow complex shapes to be created through simple instructions

Task 4

Question4.py > ...

```python
#Task 4

def startriangle(n):
    for i in range(n, 0, -1):
        print('*' * i)

print(startriangle(5))
```

```
jegremy@jegremys-MacBook python co
*****
****
***
**
*
```

In this case i made n = 5

def startriangle(n):

Prints a triangle of stars that gets smaller each line

for i in range(n, 0, -1):

the loops starts at n and n goes down by 1 every loop so if n = 4, each line would print :

4, 3 , 2, 1

Task 5

```python
#task 5

def triangular (n):
    return sum (range(1, n+1))

print(triangular(6))
```

return sum (range 1, n+1))

It creates a list of numbers from 1 to n and sums up all the numbers together

```
None
jegremy@jegremys-
21
```

Task 6

Question6.py > ...

```python

#Task 6

def fibonacci(n):
    if n == 1 or n == 2:
        return 1
    return fibonacci(n - 1) + fibonacci(n - 2)

for i in range(1, 11):
    print(fibonacci(i), end=", ")

print(fibonacci(10))
```

```
jegremy@jegremys-MacBook python
1, 1, 2, 3, 5, 8, 13, 21, 34, 55
```

if n == 1 or n == 2: return 1

It makes sure the first two fibonacci numbers are 1

for i in range(1, 11):

Prints out the first n fibonacci numbers. In this case N is 10.

Task 7

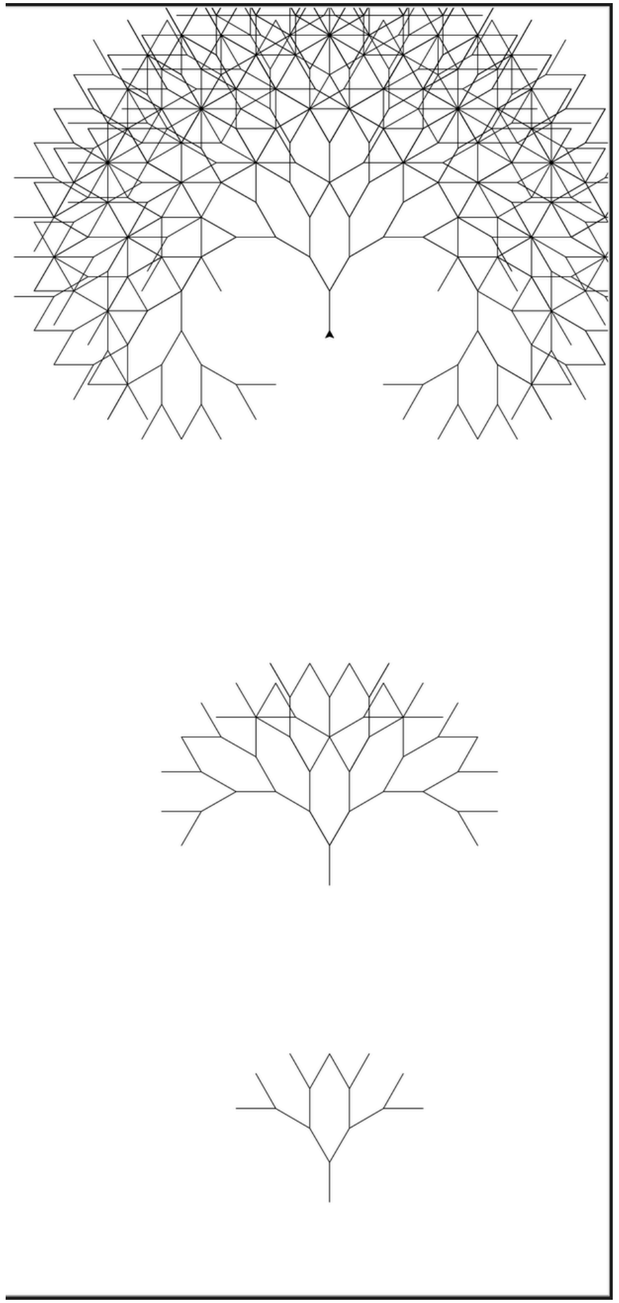
question7.py > ...

```python
import turtle
#create turtle screen and set background colour
screen=turtle.Screen()
screen.bgcolor("White")
# create a turtle object to draw with
pen=turtle.Turtle()
# set colour of the turtle
pen.color("Blue")
# set speed of the turtle
pen.speed(2)
# draw line of length 20 pixels
pen.forward(20)
# rotate the turtle 90 degrees to the right to
# change the direction it’s about to draw in
pen.right(90)
# draw the other 3 sides of a rectangle
pen.forward(40)
pen.right(90)
pen.forward(20)
pen.right(90)
pen.forward(40)
#hide turtle
pen.hideturtle()
#keep turtle window open when done
turtle.done()
```

Task 7

```python
import turtle

def simpletree(n):

    if n == 0:
        return

# [--------- Tree drawing ---------]
    turtle.forward(50)
    turtle.left(30)
    simpletree(n - 1)
    turtle.right(60)
    simpletree(n - 1)
    turtle.left(30)
    turtle.backward(50)


# [--------- Test ---------]
turtle.speed(0)

# [--------- Tree Level 4 ---------]
turtle.penup()
turtle.goto(-700, -150)
turtle.pendown()
simpletree(4)
```

```python
# [------- Tree Level 6 -------]
turtle.penup()
turtle.goto(-300, -150)
turtle.pendown()
simpletree(6)

# [-------- Tree Level 10 ---------]
turtle.penup()
turtle.goto(400, -150)
turtle.pendown()
simpletree(10)

turtle.done()
# [---------- Running the code ------------]
simpletree
```



if n == 0:
return
makes sure when n = 0, the code stops

Different positions for the trees

turtle.foward(50)
turtle.left(30)
simpletree(n-1)
turtle.right(60)
simpletree(n-1)
turtle.left(30)
turtle.backward(50)

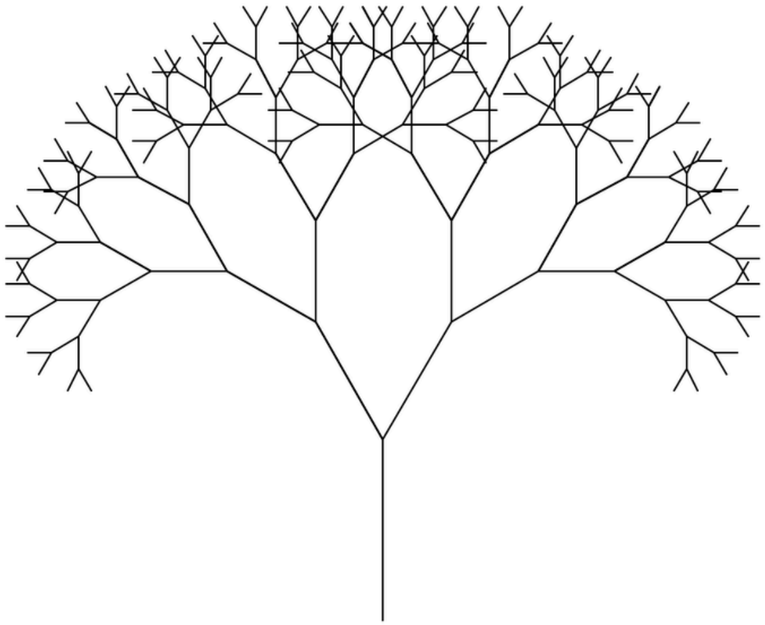
draws a branch and smaller one after

simpletree(n-1)

After branch is drawn, draws the other level tree. For example: n = 4. The next tree drawn will be tree 3.

Task 8

```python
import turtle

#[-------- Values --------]
n = 8
length = 100
angle  = 30
scale  = 0.75

#[-------- Fractal Tree --------]

def fractaltree(n, length, angle, scale):
    if n == 0:
        return
    turtle.forward(length)
    turtle.left(angle)
    fractaltree(n - 1, length * scale, angle, scale)
    turtle.right(2 * angle)
    fractaltree(n - 1, length * scale, angle, scale)
    turtle.left(angle)
    turtle.backward(length)

#[-------- Turtle Settings --------]
turtle.tracer(False)
turtle.speed(0)
turtle.penup()
turtle.goto(0, -200)
turtle.setheading(90)
turtle.pendown()

fractaltree(n, length, angle, scale)

#[-------- Ending the tree, im using a different once because the old one was weird ---------]

turtle.exitonclick()
```



Assigned values for length, angle, tree level and scale

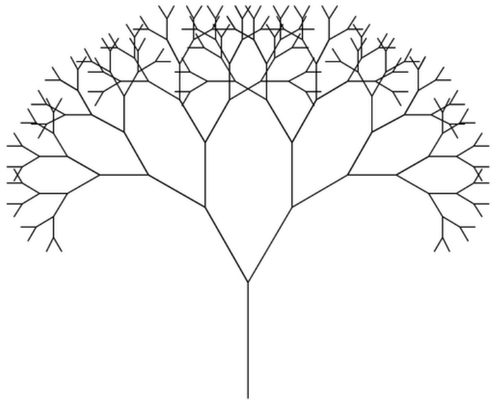
N is the tree level
Length is the length of each branch in pixels
Angle is the angle of each branch
Scale is the size of the entire tree

Fractaltree(n-1, length * scale, angle, scale) draws a smaller branch with the inputted length, scale and angle

Task 9

Similar to question 8, just added a menu

```python
import turtle

#[-------- Determining the values --------]
n = int(input("tree level "))
length = int(input("length pixels "))
angle  = int(input("angle between branches "))
scale  = (float(input("scale or size ")))

#[-------- Fractal Tree --------]

def fractaltree(n, length, angle, scale):
    if n == 0:
        return
    turtle.forward(length)
    turtle.left(angle)
    fractaltree(n - 1, length * scale, angle, scale)
    turtle.right(2 * angle)
    fractaltree(n - 1, length * scale, angle, scale)
    turtle.left(angle)
    turtle.backward(length)

#[-------- Turtle Settings --------]
turtle.tracer(False)
turtle.speed(0)
turtle.penup()
turtle.goto(0, -200)
turtle.setheading(90)
turtle.pendown()

fractaltree(n, length, angle, scale)

#[-------- Ending the tree, im using a different once because the old one was weird ---------]

turtle.exitonclick()
```



scale = float(input(“scale or size “)))

Used float because it allows me to use decimals

int(input(“tree level “)))

Creates a menu where i can type inputs

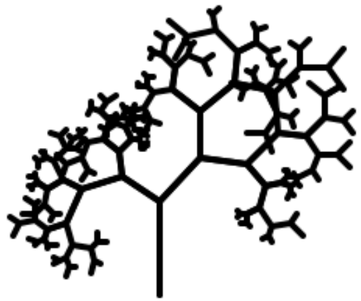
```
jegremy@jegremys-MacBook python cod
tree level 8
length pixels 100
angle between branches 30
scale or size 0.75
```

Task 10

```python
import turtle
import random

#[-------- Determining the values --------]
level = int(input("tree level "))
angle = int(input("length pixels "))
length = int(input("angle between branches "))
scale = float(input("scale size "))

#{-------- Turtle setup and settings --------]
screen = turtle.Screen()
screen.setup(width=800, height=600)


t = turtle.Turtle()
t.left(90)
t.speed(0)
t.pensize(2)
t.hideturtle()

t.penup()
t.goto(0, -100)
t.pendown()
#[-------- fractal tree random ---------]
```

```python
def randomfractaltree(n, length, angle, scale):

    if n == 0:
        return

    t.forward(length)


    angle_variation = angle * 0.20
    scale_variation = scale * 0.90


    random_angle_left = angle / 2 + (random.random() - 0.5) * angle_variation
    random_scale_left = scale + (random.random() - 0.5) * scale_variation
    random_scale_left = max(0.5, min(0.9, random_scale_left))


    t.left(random_angle_left)
    randomfractaltree(n - 1, length * random_scale_left, angle, scale)
    t.right(random_angle_left)


    random_angle_right = angle / 2 + (random.random() - 0.5) * angle_variation
    random_scale_right = scale + (random.random() - 0.5) * scale_variation
    random_scale_right = max(0.5, min(0.9, random_scale_right))


    t.right(random_angle_right)
    randomfractaltree(n - 1, length * random_scale_right, angle, scale)
    t.left(random_angle_right)


    t.backward(length)
```

```python
print(f"Drawing tree")

#[-------- Randomness ---------]

turtle.tracer(0,0)
randomfractaltree(level, length, angle, scale)
turtle.update()
turtle.done()
```



T.left(random_angle_left)
goes back to original position
randomfractaltree(n-1, length * random_scale_left, angle, scale)
Creates a smaller branch
angle_variation = angle * 0.2
scale_variation = scale * 0.90
Creates a range of 20% and 90% of the angle and scale to randomise branches.
Random_angle_left = angle . 2 + (random,random() - 0.5) * angle)variation
Creates a random left angle and adds random amounts to the angle.
random.random
gives you a random number from -0.5 to +0.5